

# One combined presentation covering all 10 official DVSA vulnerabilities, exploit logic, remediation, and post-fix verification.

Built from the local final lesson 5 and 10 reports plus the cloned GitHub lesson notes for the remaining official lessons.

## LESSON 1

Event Injection makes the order

## LESSON 2

Broken JWT validation lets one

## LESSON 3

Receipt data becomes reachable once

## LESSON 4

A public-write S3 bucket becomes

## LESSON 5

The public /order path should

## LESSON 6

Parallel billing requests can deny

## LESSON 7

An over-privileged Lambda role turns

## LESSON 8

A race condition survives normal

## LESSON 9

Vulnerable dependencies recreate RCE even

## LESSON 10

Unhandled exceptions become information leaks

## ARCHITECTURE LENS

S3 website -> Cognito -> API Gateway -> Lambda -> DynamoDB / S3 / SQS / SES

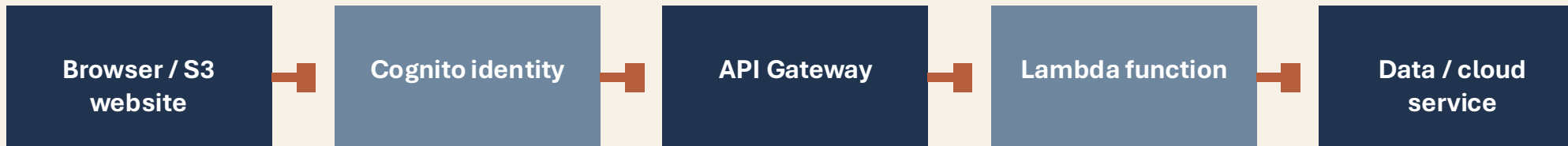
## MARKING LENS

Each lesson is framed with intended logic, exploit path, proof, fix, and verification.

## DEEP EVIDENCE

Lessons 5 and 10 use the final local report wording, code packs, and organized screenshots.

**Full marks come from showing the serverless flow, the break point, the fix, and proof that normal behavior still works.**



**Required analysis on every lesson**

- Explain the intended request path and the AWS services involved.
- Show where the trust boundary or logic breaks.
- Provide exploit evidence, then the exact fix or configuration change.
- Retest the exploit and confirm normal functionality still works.

**Security principles repeated across the project**

- Validate structure, type, and state transitions before business logic.
- Verify identity before trusting JWT claims or admin context.
- Keep Lambda roles and cloud policies least-privilege.
- Fail safely: block abuse, return controlled errors, and avoid leaking internals.

# Event Injection makes the order endpoint execute attacker-supplied code.

## ROOT CAUSE

DVSA-ORDER-MANAGER used node-serialize to unserialize attacker-controlled request data, so a serialized function marker was treated as executable JavaScript.

## Intended logic

- API Gateway should pass plain JSON into DVSA-ORDER-MANAGER.
- The handler should validate fields and treat action values as data only.

## Exploit path

- Send an `__$ND_FUNC__$` payload through POST /order.
- Injected JavaScript runs inside Lambda before the normal request path finishes.

## Fix / mitigation

- Remove unsafe deserialization and parse only JSON.
- Reject malformed structures before business logic continues.
- Replay the same payload and confirm no injected log message or backend code execution appears in CloudWatch.

## AFFECTED COMPONENTS

API Gateway, DVSA-ORDER-MANAGER, CloudWatch Logs

## SECURE DESIGN PRINCIPLE

Input validation + removal of unsafe evaluation

## VERIFICATION FOCUS

Replay the same payload and confirm no injected log message or backend code execution appears in CloudWatch.

# Broken JWT validation lets one user read another user's orders.

## ROOT CAUSE

The backend trusted decoded JWT claims such as username and sub without verifying the token signature and issuer properly.

### Intended logic

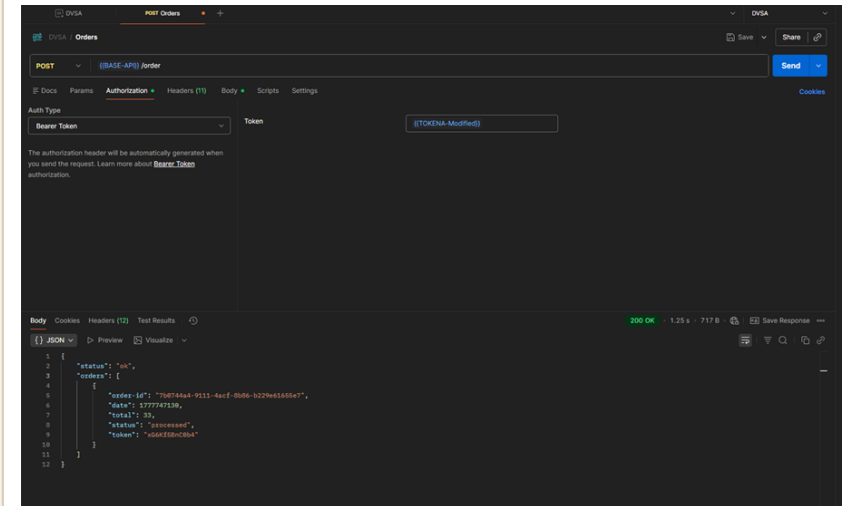
- Orders requests should return only the authenticated user's data.
- Identity claims should be trusted only after signature verification.

### Exploit path

- Decode a normal user's token, replace username and sub with victim values, then re-encode it.
- Reuse the forged token on the orders action to retrieve the victim's order list.

### Fix / mitigation

- Verify JWT signature and claims server-side on every protected request.
- Reject forged or altered tokens before authorization decisions.
- After the fix, the forged token is rejected and a normal valid token still returns only the attacker's own orders.



Repo proof image showing forged-token access to victim order data.

## AFFECTED COMPONENTS

API authentication layer, orders endpoint

## Receipt data becomes reachable once RCE is chained with missing authorization.

## ROOT CAUSE

Unsafe deserialization created RCE in the order path, and an admin receipt function lacked strong backend authorization checks around receipt access.

**Intended logic**

- Receipt generation and download links should stay behind an explicit admin boundary.
- The public order path should never reach receipt data workflows directly.

### Exploit path

- Trigger malicious code through /order and invoke privileged receipt behavior indirectly.
- Capture exfiltrated receipt data or signed URLs through an attacker-controlled webhook.

### Final verification

- Remove the unsafe deserialization entry point.
- Enforce role checks on receipt-generation and receipt-access functions.
- After remediation, the same payload no longer executes code and privileged receipt data is not exposed to a non-admin context.

[illegible]

CloudWatch proof from the repo showing backend execution during the exploit chain.

DVSA-ORDER-MANAGER, admin receipt Lambda, S3 receipt storage

# A public-write S3 bucket becomes an execution path when Lambda trusts uploaded objects.

## ROOT CAUSE

The feedback bucket allowed public or cross-account writes, and downstream Lambda processing trusted attacker-controlled filenames and object metadata.

## Intended logic

- Only authorized actors should upload objects into DVSA-controlled storage.
- Backend processors should sanitize filenames and never treat them as executable input.

## Exploit path

- Upload a maliciously named object into the public bucket.
- Wait for Lambda to process the object name and carry the injected content into backend execution or logs.

## Fix & validation

- Tighten S3 bucket policy and remove unintended write access.
- Validate uploaded object names and isolate storage from execution logic.
- After the fix, unauthorized uploads fail and no attacker-controlled filename reaches the backend processing path.

```

> 2020-05-04T14:50:02.261+00:00 START RequestID: c7d08000-704a-470c-b03c-7898384043b0 Version: $LATEST
> 2020-05-04T14:50:02.264+00:00 ["Records": [{"eventVersion": "1.0", "eventSource": "aws:s3", "awsRegion": "us-east-1", "eventTime": "2020-05-04T14:50:01.332Z", "eventName": "ObjectCreated:Put", "userIdentity": {"principalId": "AOLAD4XK9Y3120068623L"},
> 2020-05-04T14:50:02.264+00:00 null
> 2020-05-04T14:50:02.264+00:00 x.foo /tmp/mull
> 2020-05-04T14:50:02.264+00:00 null
> 2020-05-04T14:50:02.264+00:00 x.foo.s3
> 2020-05-04T14:50:02.375+00:00 END RequestID: c7d08000-704a-470c-b03c-7898384043b0
> 2020-05-04T14:50:02.375+00:00 REPORT RequestID: c7d08000-704a-470c-b03c-7898384043b0 Duration: 134.42 ms Billed Duration: 135 ms Memory Size: 128 MB Max Memory Used: 58 MB

```

Repo screenshot used as exploit/configuration proof for the S3 lesson.

## AFFECTED COMPONENTS

Amazon S3 bucket policy, feedback-processing Lambda, CloudWatch

# The public /order path should never be able to force a paid-state admin update.

## ROOT CAUSE

A serialized payload sent to /order could invoke DVSA-ADMIN-UPDATE-ORDERS indirectly, and the admin function lacked a strong admin-only authorization gate and state allowlist.

## Intended logic

- Normal users must complete new -> shipping -> billing through the public workflow.
- Only trusted backend/admin logic should move an order into the paid state.

## Exploit path

- Create a normal Super Mario 3 order and complete the shipping step.
- Generate /tmp/vuln5\_order\_injection.json and replay it through /order to reach DVSA-ADMIN-UPDATE-ORDERS indirectly.

## Fix + verification

- Replace unsafe parsing in order-manager.js.
- Require an admin-only gate in admin\_update\_orders.py and block forged paid-state transitions with a status allowlist.
- Post-fix replay returns {"status": "err", "msg": "unknown action"}; DynamoDB stays unchanged and the normal order flow still works.

Table: DVSA-ORDERS-DB - Items returned (1/11)

Scan started on May 05, 2026, 22:27:05

| orderid (String)       | userid      | address       | confirmationT...    | ItemList                 | orderStatus | payment... | totalAmount |
|------------------------|-------------|---------------|---------------------|--------------------------|-------------|------------|-------------|
| 51fd0f32-e7f1-43a7-... | 34084408... | ["name": {... | forged-confirmation | ["Rampart": { "N": ...   | paid        | 0          | 48          |
| 65d579e-0645-465d-...  | 34084408... | ["name": {... | forged-confirmation | ["Super Mario 3": ...    | paid        | 0          | 35          |
| 23f6b0c3-79ea-40c3-... | 34084408... | ["name": {... | forged-confirmation | ["Rampart": { "N": ...   | paid        | 0          | 48          |
| a39eade9-7752-4a6f-... | 34084408... | ["name": {... | forged-confirmation | ["Rampart": { "N": ...   | paid        | 0          | 48          |
| a74ac8ef-e9ba-40a2-... | 34084408... | ["name": {... | nLINKUK3qB          | ["1020": { "N": "1": ... | 200         | 1777796835 | 25          |
| 33c7e1e1-4300-4755-... | 34084408... | ["name": {... | 665NkudQub          | ["1010": { "N": "1": ... | 200         | 1777836661 | 28          |
| 3a183ee1-a280-4423-... | 34084408... | ["name": {... | 17J2hvc02D          | ["1018": { "N": "1": ... | 200         | 1777794836 | 33          |
| 6a074cc-e700-4933-...  | 34084408... | ["name": {... | uk8Qz9W0Kaa         | ["1010": { "N": "1": ... | 200         | 1777793154 | 35          |
| 265de950-9980-4c38-... | 34084408... | ["name": {... |                     | ["1020": { "N": "1": ... | 100         | 1777795806 | 0           |
| 6b6de0ff-ba08-481b-... | 34084408... | ["name": {... |                     | ["1020": { "N": "1": ... | 100         | 1777796707 | 0           |
| e0604095-3a79-43a5-... | 34084408... | ["name": {... |                     | ["1020": { "N": "1": ... | 100         | 1777796765 | 0           |

Pre-fix proof

```
{
  "status": "err",
  "msg": "unknown action"
}
MacBook-Pro-8:~ mac$
MacBook-Pro-8:~ mac$ curl -i -s "SAPI" \
> -H "content-type: application/json" \
> -H "authorization: $TOKEN" \
> --data-binary @/tmp/vuln5_order_injection.json
HTTP/2 200
content-type: application/json
content-length: 39
date: Sun, 03 May 2026 19:59:31 GMT
x-amzn-trace-id: Root=1-69f7a923-213a7809484073aa457dic14;Parent=61aa6b7016d6e2f6;Sampled=0;Lineage=1:62672913:0
x-amzn-requestid: 6d799428-e13e-46ab-b83a-c440fde53638
access-control-allow-origin: *
x-amz-apigw-id: czddmE-rIAMElKw=
x-cache: Miss from cloudfront
via: 1.1 668d437438b329db5ce80bd2f5d5325e.cloudfront.net (CloudFront)
x-amz-cf-pop: JED60-PS
x-amz-cf-id: PL7hZJcA2Y0hYy0FGRA83zOK6ZHm7UAXdAuQtWu0jkcFP9uLVNvaA==
{"status": "err", "msg": "unknown action"}
MacBook-Pro-8:~ mac$
```

Post-fix verification

# Parallel billing requests can deny service when the API has no fairness controls.

## ROOT CAUSE

The billing path accepted unlimited concurrent requests, so repeated parallel submissions exhausted processing capacity and surfaced internal failures to legitimate users.

## Intended logic

- Billing should stay available for real users even under abusive traffic.
- The API layer should throttle or queue bursts before backend resources are exhausted.

## Exploit path

- Replay hundreds of billing requests in parallel against /order.
- Observe repeated internal server errors and degraded checkout availability.

## Fix verification

- Configure API Gateway throttling and rate controls.
- Limit burst capacity and keep critical resources stable under abuse.
- After the fix, abusive replay yields Too Many Requests instead of backend collapse, while normal billing still succeeds.

## AFFECTED COMPONENTS

API Gateway, billing Lambda path

## SECURE DESIGN PRINCIPLE

Abuse resistance through throttling and resource protection

## VERIFICATION FOCUS

After the fix, abusive replay yields Too Many Requests instead of backend collapse, while normal billing still succeeds.



# An over-privileged Lambda role turns any compromise into wider AWS compromise.

## ROOT CAUSE

DVSA-SEND-RECEIPT-EMAIL ran with wildcard S3 and DynamoDB permissions plus broad SES access, far beyond what the function actually needed.

### Intended logic

- A Lambda role should expose only the exact actions and resources the function needs.
- Compromising one function should not automatically grant account-wide data access.

### Exploit path

- Use IAM Policy Simulator to prove the role can touch unrelated S3 buckets and DynamoDB tables.
- Compare that breadth with CloudTrail-generated least-privilege evidence from actual runtime use.

### Fix + verification

- Replace broad managed/custom policies with one narrow least-privilege policy.
- Allow only required log, decrypt, email, and specific data-access actions.
- Post-fix simulator tests deny unrelated S3 and DynamoDB access, while the normal receipt workflow still sends email successfully.

## AFFECTED COMPONENTS

DVSA-SEND-RECEIPT-EMAIL Lambda role, IAM policies, CloudTrail

## SECURE DESIGN PRINCIPLE

Least privilege

## VERIFICATION FOCUS

Post-fix simulator tests deny unrelated S3 and DynamoDB access, while the normal receipt workflow still sends email successfully.

# A race condition survives normal checks until the final write becomes atomic.

## ROOT CAUSE

DVSA-ORDER-BILLING checked order state once, then charged payment before an unconditional DynamoDB update, leaving a TOCTOU window for cart changes during billing.

## Intended logic

- The billed quantity and charged amount should stay locked to the same committed order state.
- Concurrent requests should not let a user receive more items than they paid for.

## Exploit path

- Intercept checkout traffic and race a cart update into the gap between validation and final write.
- Receive more items while paying the earlier lower amount.

## Fix / mitigation

- Add a DynamoDB ConditionExpression so the state check and update happen atomically.
- Let the database enforce the invariant at write time.
- After remediation, the raced update fails and a normal checkout still completes with the correct receipt.

## AFFECTED COMPONENTS

DVSA-ORDER-BILLING, DynamoDB orders state

## SECURE DESIGN PRINCIPLE

Atomic state transition

## VERIFICATION FOCUS

After remediation, the raced update fails and a normal checkout still completes with the correct receipt.

# Vulnerable dependencies recreate RCE even when the bug appears to be 'just a library issue.'

## ROOT CAUSE

The `node-serialize` dependency accepted serialized functions, so attacker-controlled input became executable during deserialization in the same order-processing path.

### Intended logic

- Third-party packages should never expand JSON parsing into code execution.
- Dependency choice is part of the application attack surface.

### Exploit path

- Send a serialized-function payload through `/order`.
- Confirm the injected marker appears in CloudWatch logs before the normal handler fails.

### Fix verification

- Remove the vulnerable dependency and parse only standard JSON.
- Keep input structure validation in front of the business logic.
- After the fix, the same payload no longer produces the injected CloudWatch marker or eval path.

## AFFECTED COMPONENTS

DVSA-ORDER-MANAGER, `node-serialize` dependency, CloudWatch Logs

## SECURE DESIGN PRINCIPLE

Supply-chain hygiene + safe parsing

## VERIFICATION FOCUS

After the fix, the same payload no longer produces the injected CloudWatch marker or eval path.

# Unhandled exceptions become information leaks when malformed input reaches backend code paths.

## ROOT CAUSE

get\_order.py trusted request field types, called .lower() on non-string isAdmin values, and later let malformed numeric order-id values trigger a DynamoDB ValidationException that escaped to the client.

## Intended logic

- The get action should validate request structure before touching business logic or DynamoDB.
- Client errors should return controlled messages without leaking stack traces or internal paths.

## Exploit path

- Send malformed get requests with numeric order-id values through the public endpoint.
- Observe AttributeError and ValidationException details leaking /var/task/get\_order.py and backend internals.

## Fix verification

- Add parse\_is\_admin(value), validate orderId and userId as non-empty strings, and wrap the handler in generic exception handling.
- Return a controlled invalid request response instead of leaking internal traces.
- After the fix, the same malformed request returns {"status": "err", "msg": "invalid request"}; CloudWatch shows START, END, and REPORT with no unhandled exception.

```
MacBook-Pro-8:~ mac$ curl -i -s "$API" \
> -H "Content-Type: application/json" \
> -H "Authorization: $TOKEN" \
> -data-raw '{"action": "get", "order-id": 12345}'
HTTP/2 200
content-type: application/json
content-length: 237
date: Mon, 04 May 2026 08:55:13 GMT
x-amzn-trace-id: Root=1-69f85ef0-5834070a1afb35db3ebe5e5c;Parent=22ad2cca6788acbc;Sampled=0;Lineage=1:62672913:0
x-amzn-requestid: 859a1e87-f8d6-47d6-87dc-23d2280fc3f8
access-control-allow-origin: *
x-amz-apigw-id: c1PFpFX21AMEFQ==
x-cache: Miss from cloudfront
via: 1.1 61954eaa5e77bcffc1d48963f5835828.cloudfront.net (CloudFront)
x-amz-cf-pop: MRS62-P5
x-amz-cf-id: mX9PK-CoyvN8xcKXp3GHJ31PzC-ysaefq71myC3cSLNkaDvuoog==

{"errorMessage": "bool object has no attribute 'lower'", "errorType": "AttributeError", "stackTrace": [{"File": "\n/var/task/get_order.py", "line": 32, "in": "lambda_handler\n    is_admin = json.loads(event.get('isAdmin', '\\nse\\\\').lower())\\n"}]MacBook-Pro-8:~ mac$ curl -i -s "$API" \
> -H "Content-Type: application/json" \
>
```

Pre-fix proof

The screenshot shows the AWS CloudWatch console with the 'Log events' tab selected. The log group is 'arn:aws:logs:us-east-1:123456789012:log-group:/var/task/get\_order.py'. The log entry shows a message: '{"errorMessage": "bool object has no attribute 'lower'', "errorType": "AttributeError", "stackTrace": [{"File": "\n/var/task/get\_order.py", "line": 32, "in": "lambda\_handler\n is\_admin = json.loads(event.get('isAdmin', '\\nse\\\\').lower())\\n"}]'. The log entry is from the 'START' event of a function named 'lambda\_handler'.

Post-fix verification